Queue

基本概念

Queue（佇列）可以作為突發流量的緩衝區，或作為在系統中分配工作的手段。一個計算資源會將訊息發送到 queue，然後不再關心它們。在另一端，一組 worker（同樣是計算資源）會依照自己的節奏處理這些訊息。訊息可以是任何形式，從簡單的字串到複雜的物件。

Queue 的功能是吸收突發的流量，避免系統過載。如果我突然收到 1,000 個請求，但每秒只能處理 200 個請求，那麼其中 800 個請求會先在 queue 中等待處理 —— 但它們不會被丟棄！Queue 也能將系統中的生產者（producer）與消費者（consumer）解耦，使你可以分別獨立擴展它們。我可以在 queue 後面的服務進行關閉或重啟，而幾乎不會影響整體系統。

在同步工作負載中引入 queue 時要特別小心。如果你有強烈的延遲要求（例如 < 500ms），一旦加入 queue，幾乎可以確定會破壞這個延遲限制。

架構特徵

- **Producer (生產者)**：將訊息送進 queue
- **Queue (佇列)**：存放訊息，通常支援持久化
- **Consumer (消費者)**：從 queue 中取出訊息處理

特點：

- **非同步 (Asynchronous)**
  - Producer 將訊息送進 Queue 後，不需要等待 Consumer 立即處理。

buildmoat.org

    - 範例：使用者上傳圖片 → API 先快速回覆成功 → 圖片壓縮工作交給 Queue，稍後由背景 Worker 處理。
    - 好處：提升用戶體驗（降低等待時間）、減少前端阻塞。
- **解耦 (Decoupling)**
    - Producer 不需要知道 Consumer 是誰、在哪裡、是否在線。
    - Consumer 也不需要關心 Producer 的細節，只需從 Queue 取訊息。
    - 範例：訂單系統產生訂單訊息 → 發送到 Queue → Email Service、Inventory Service 都可以各自消費。
    - 好處：降低系統之間的依賴，讓服務能獨立開發與擴展。
- **彈性 (Elasticity)**
    - 當流量高峰時，訊息會堆積在 Queue 中；Consumer 可以動態擴展來消化訊息。
    - 當流量降低時，Consumer 數量也可以縮減，節省資源。
    - 範例：黑色星期五購物高峰，Queue 接住暴增的訂單流量，後端服務用多個 Consumer 節點分批處理。
    - 好處：能「削峰填谷」，讓系統穩定運作。

常見功能

- **持久化 (Persistence)**：訊息可存到磁碟，避免丟失
- **重試（Retry）與死信佇列 (Dead Letter Queue, DLQ)**：處理失敗訊息
- **延遲與排程 (Delay / Scheduled Messages)**：控制訊息什麼時候被處理
- **順序性 (Ordering)**：確保同一 partition 的訊息順序一致
- **多消費者模型**：支援多個消費者同時處理訊息

常見產品與服務

- **雲端服務**
    - AWS SQS
    - Google Pub/Sub
    - Azure Queue Storage

buildmoat.org

- **開源解決方案**
  - RabbitMQ（AMQP 協定）
  - Kafka（偏向高吞吐量 log stream）
  - ActiveMQ
  - Redis Stream

適用場景

- **任務非同步化**
  - 在一個雲端照片處理服務中，queue 可用來分配昂貴的影像處理任務。當使用者上傳照片進行編輯或濾鏡處理時，這些任務會被放入 queue。不同的 worker 節點會從 queue 拉取任務，確保工作負載平均分配並有效利用計算資源。
- **流量削峰**
  - 在像 Uber 這樣的叫車應用中，queue 可用來管理突然激增的叫車請求。在高峰時段或特殊活動期間，叫車請求可能會大量暴增。Queue 會緩衝這些進來的請求，使系統能以可控的速率處理它們，而不會讓伺服器過載或影響使用者體驗。
- **服務解耦**
  - 訂單系統與寄信系統透過 queue 連接，互不影響
- **事件驅動架構**
  - 使用 queue 作為 event bus，觸發下游服務

在系統設計面試中如何談 Queue

常見用法：

- 當你需要 **非同步處理 / 解耦 / 削峰** 時，再把 Queue 引入
- 範例：上傳圖片後立即回覆成功，實際壓縮工作丟進 Queue，由後端異步處理

面試時你應該了解的 queue 重點

- **訊息順序（Message Ordering）：**大多數 queue 是 FIFO（先進先出），也就是訊息會按照接收順序處理。然而，有些 queue（例如 Kafka）允許更複雜的順序保證，例如基於指定優先級或時間進行排序。

buildmvp.org

- **重試機制（Retry Mechanisms）**：許多 queue 內建重試機制，會在判定失敗前嘗試重新投遞訊息一定次數。你可以配置重試策略，包括每次嘗試之間的延遲，以及最大嘗試次數。
- **死信佇列（Dead Letter Queues）**：死信佇列用於儲存無法被處理的訊息。它們對於除錯與稽核非常有用，因為可以檢視那些處理失敗的訊息，並理解失敗原因。
- **透過分區進行擴展（Scaling with Partitions）**：Queue 可以分區到多台伺服器上，以擴展處理更多訊息的能力。每個分區可以由不同的一組 worker 處理。就像資料庫一樣，你需要指定 partition key，以確保相關訊息存放在同一個分區中。
- **反壓（Backpressure）**：Queue 最大的問題在於它會讓系統過載變得更容易。如果我的系統每秒只能支援 200 個請求，但實際上每秒收到 300 個請求，那麼我永遠處理不完！Queue 只是掩蓋了系統容量不足的問題。解法是 backpressure。Backpressure 是在 queue 過載時減緩訊息產生速度的一種方式。這有助於防止 queue 成為系統瓶頸。例如，如果 queue 已滿，你可能會拒絕新的訊息，或降低接受新訊息的速率，可能向使用者或生產者回傳錯誤。

buildmoat.org